

Topic 7 卷积神经网络

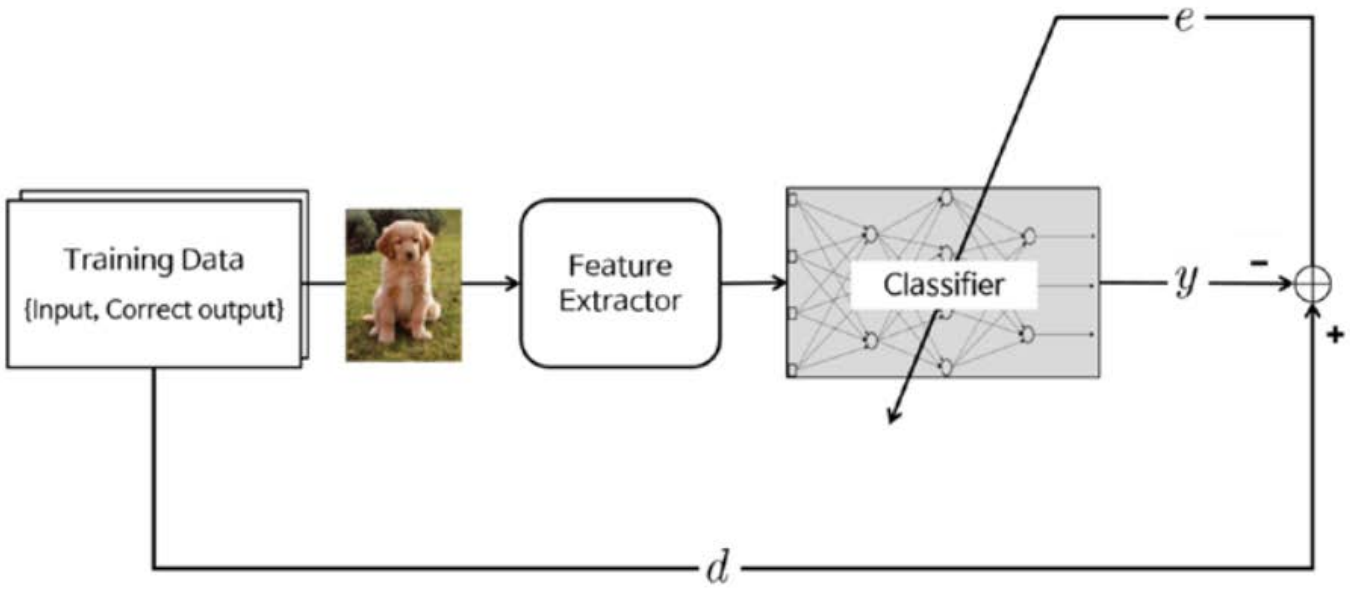
魏准
浙江大学，信息与电子工程学院
2026

总体内容

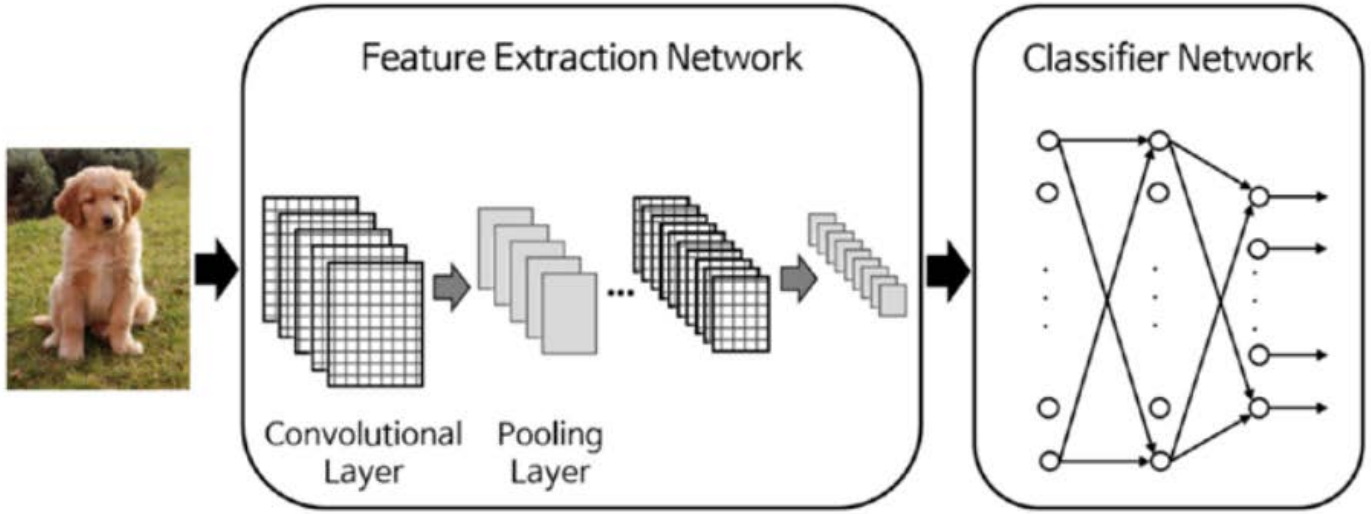
主要知识点

- 卷积层(Convolution Layer)
- 池化层(Pooling Layer)
- 实例：训练、测试一个卷积神经网络

Phil Kim, 2017. MATLAB Deep Learning: With Machine Learning, Neural Networks and Artificial Intelligence.

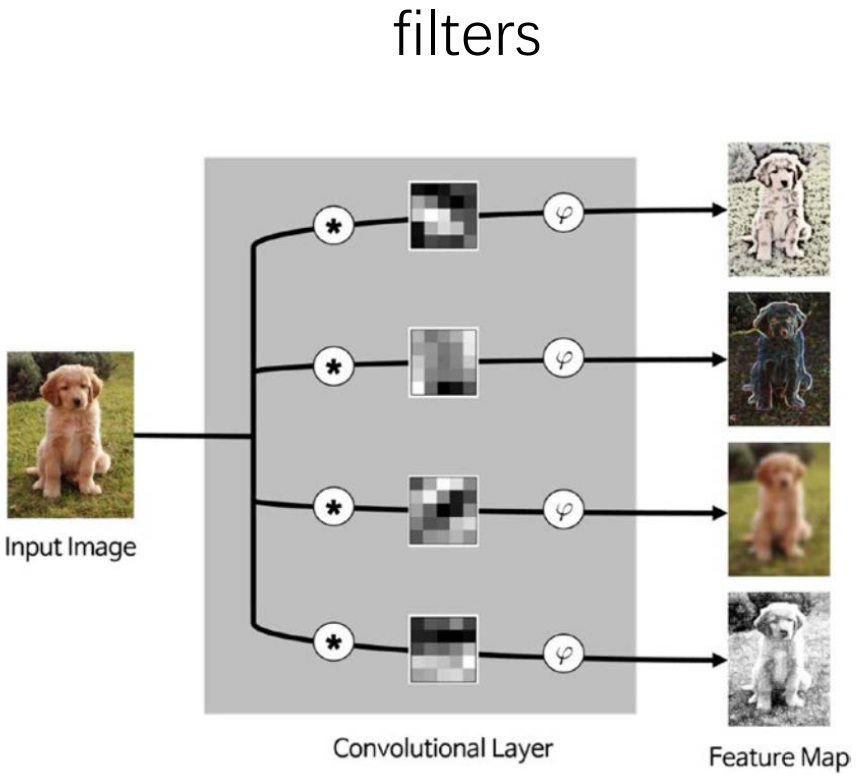


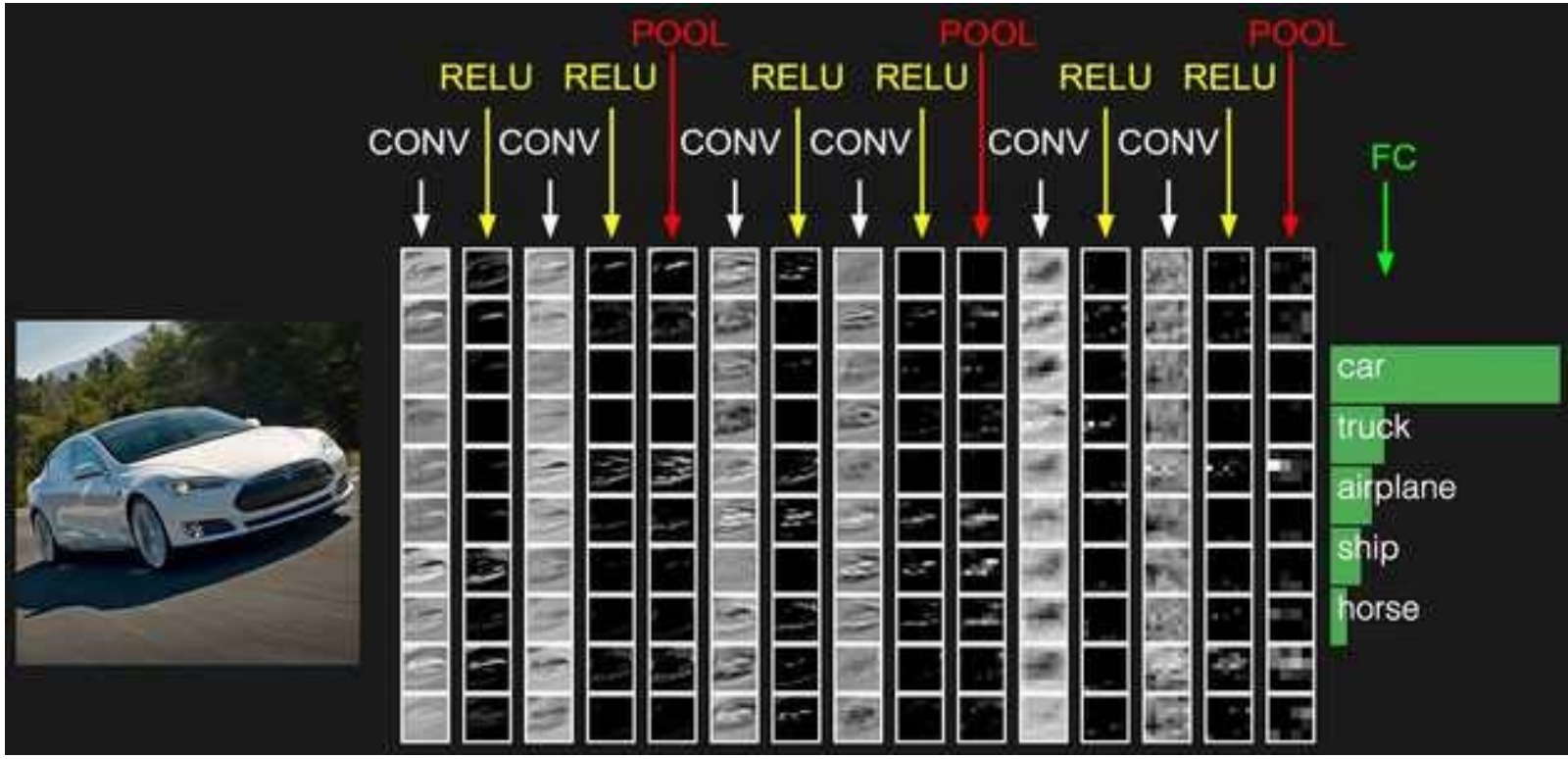
特征提取用于分类



利用卷积实现特征提取并分类

1.1：卷积层(Convolution Layer)





卷积层的基本效果

基本特征

- 输入X： *W1×H1×D1*； （长宽通道）

filters： *F×F×D1×K;*

步长Stride: *S*;

补零数zero padding number: *P;*

- 输出： W2×H2×K;

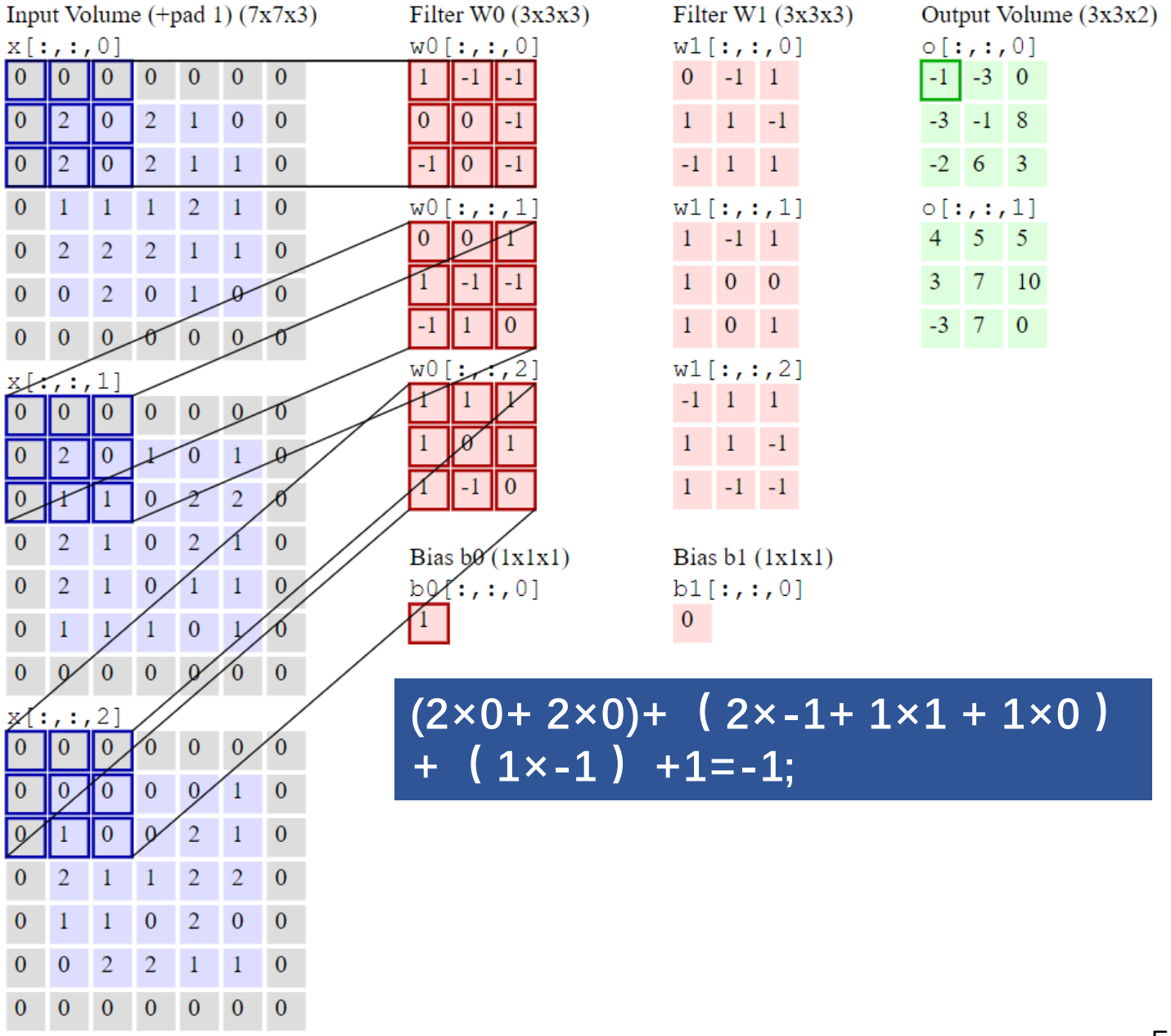
W2=(W1−F+2P)/S+1

=(5-3+2×1)/2+1=3;

H2=(H1−F+2P)/S+1

=(5-3+2×1)/2+1=3;

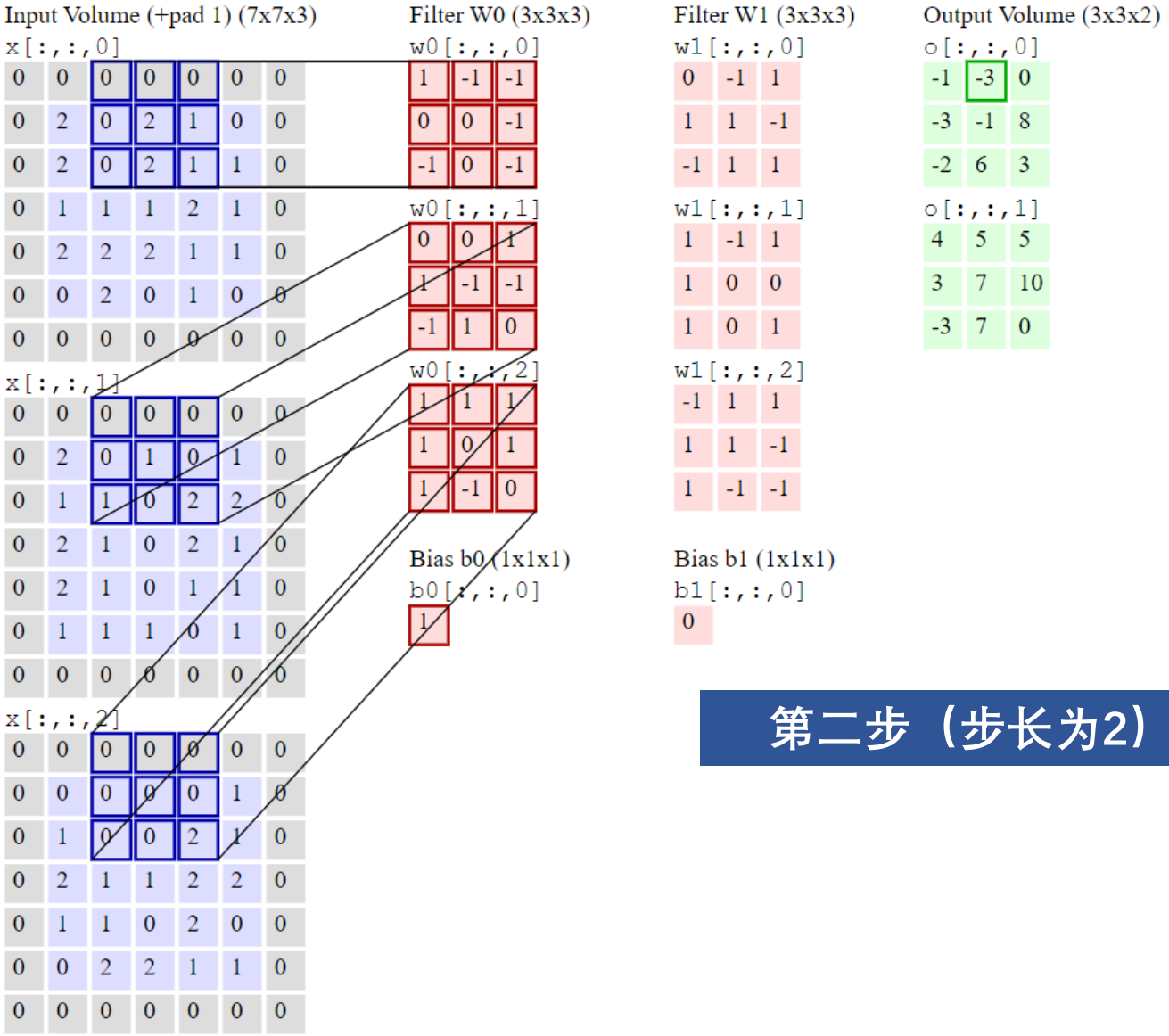
K=2;



(2×0+ 2×0)+（2×-1+ 1×1 + 1×0）+（1×-1）+1=-1;

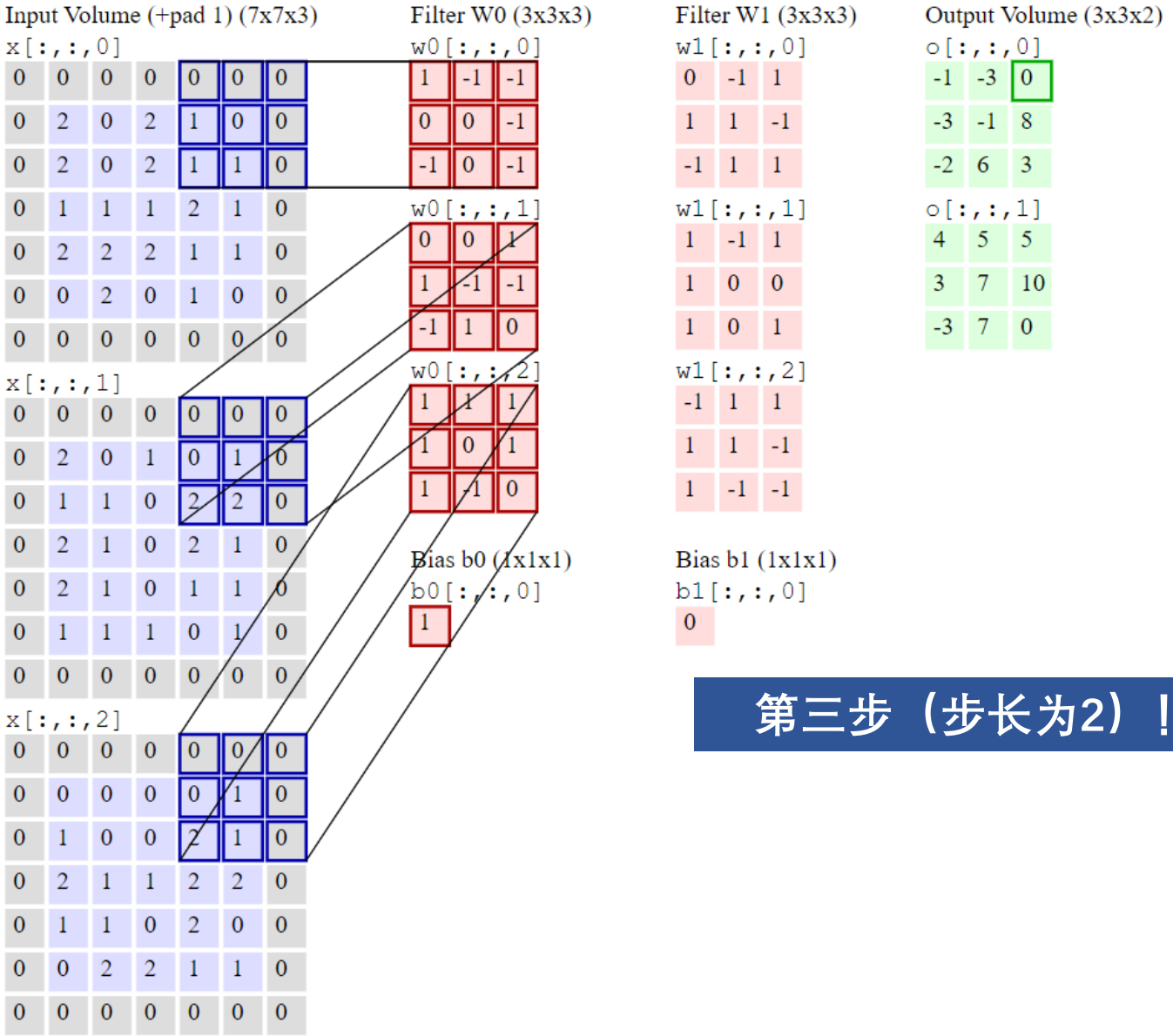
Source(CS231n: Deep Learning for Computer Vision)

基本特征



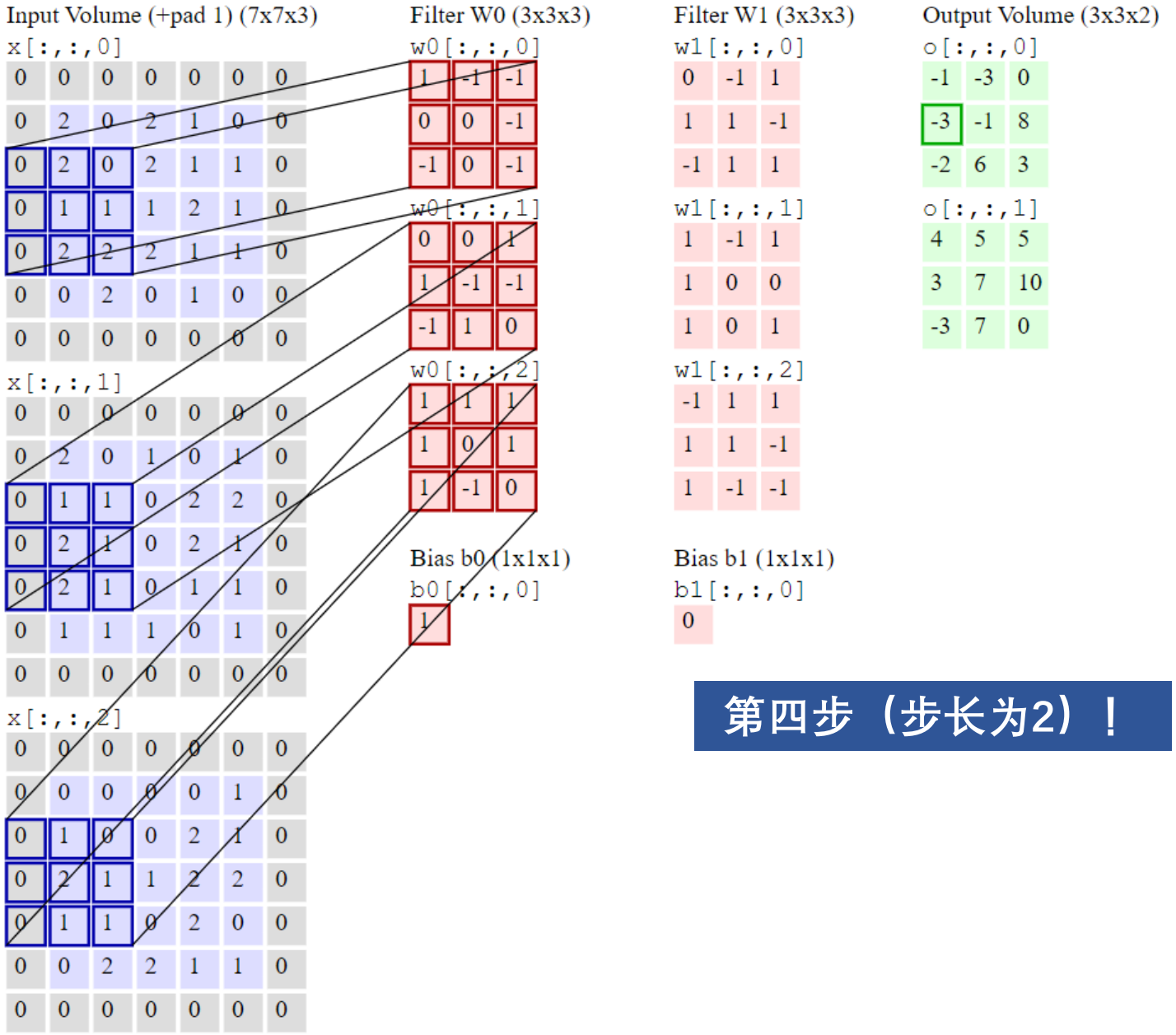
第二步（步长为2）！

基本特征



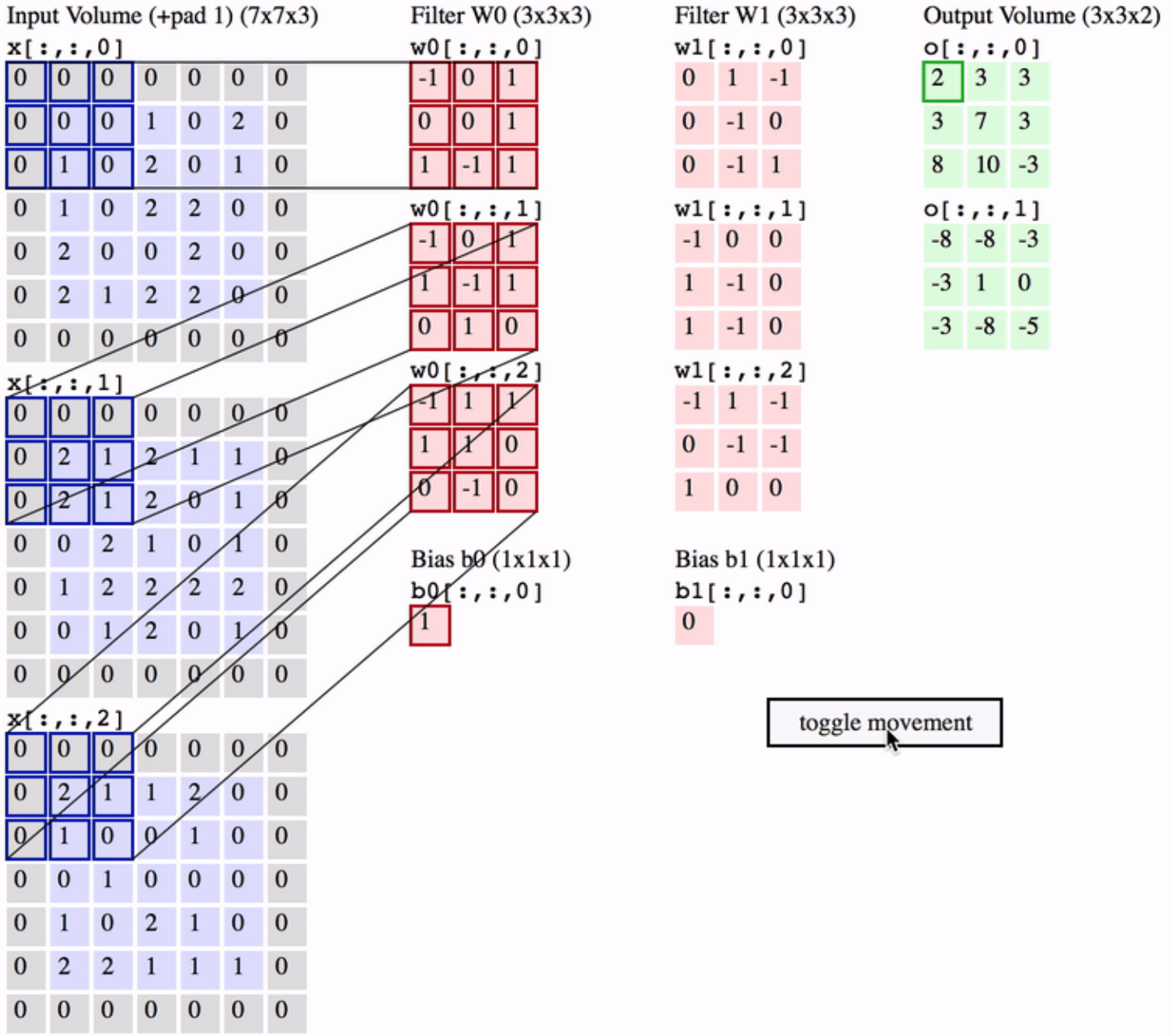
第三步（步长为2）！

基本特征

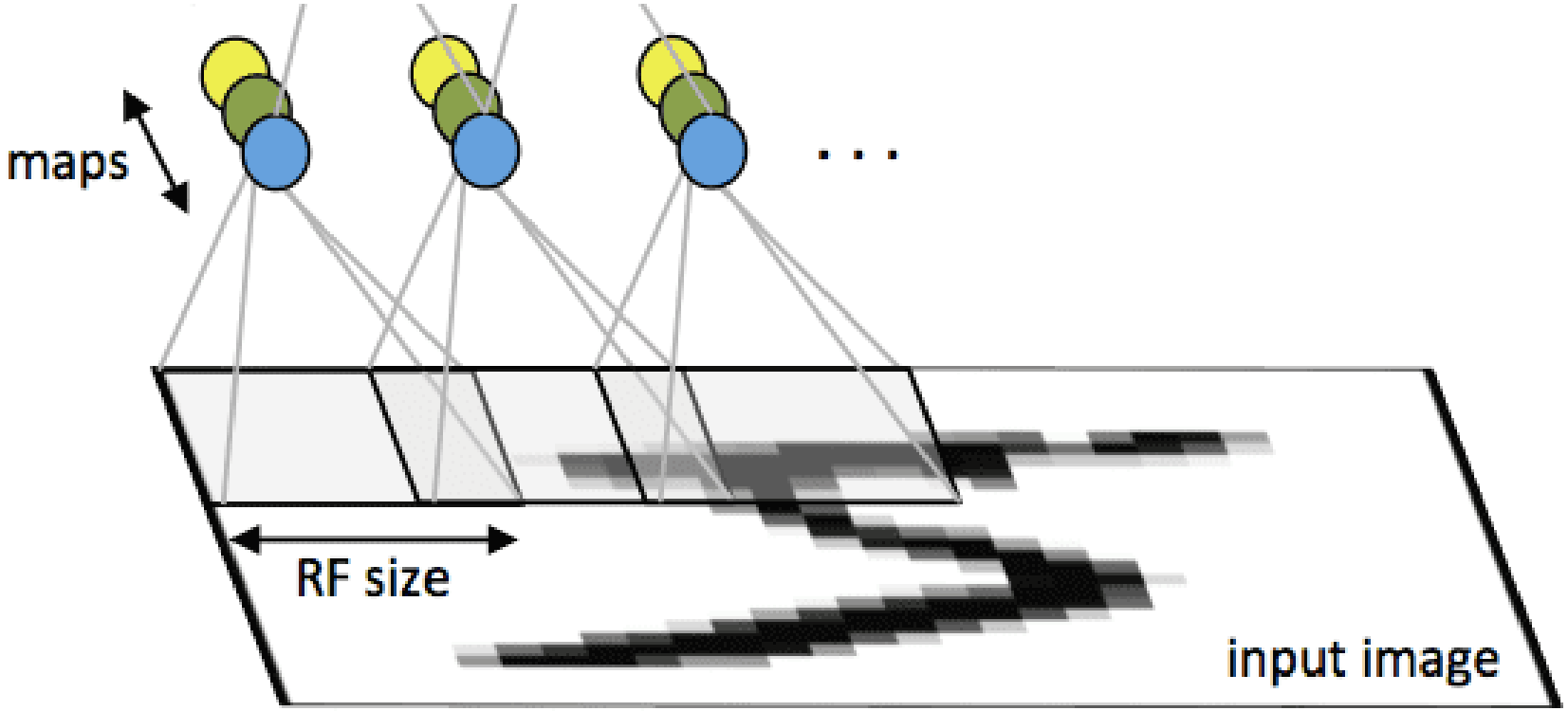


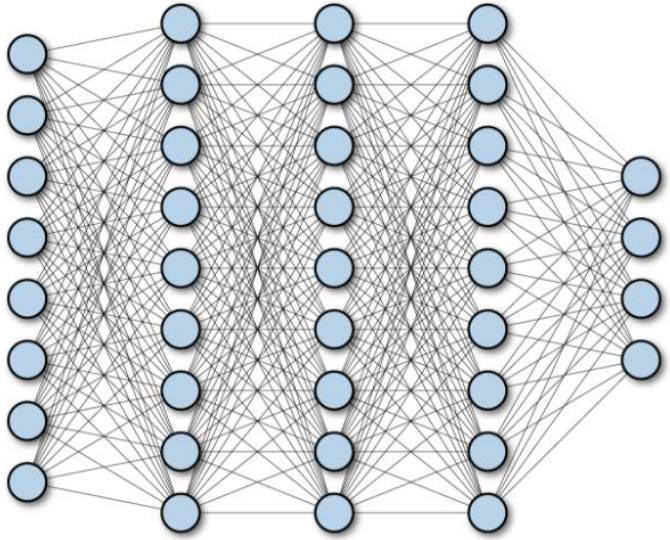
第四步（步长为2）！

基本特征


Input Volume (+pad 1) (7x7x3)
x[:,:,0]
0 0 0 0 0 0 0
0 0 0 1 0 2 0
0 1 0 2 0 1 0
0 1 0 2 2 0 0
0 2 0 0 2 0 0
0 2 1 2 2 0 0
0 0 0 0 0 0 0
x[:,:,1]
0 0 0 0 0 0 0
0 2 1 2 1 1 0
0 2 1 2 0 1 0
0 0 2 1 0 1 0
0 1 2 2 2 2 0
0 0 1 2 0 1 0
0 0 0 0 0 0 0
x[:,:,2]
0 0 0 0 0 0 0
0 2 1 1 2 0 0
0 1 0 0 1 0 0
0 0 1 0 0 0 0
0 1 0 2 1 0 0
0 2 2 1 1 1 0
0 0 0 0 0 0 0
Filter W0 (3x3x3)
w0[:,:,0]
-1 0 1
0 0 1
1 -1 1
w0[:,:,1]
-1 0 1
1 -1 1
0 1 0
w0[:,:,2]
-1 1 1
1 1 0
0 -1 0
Bias b0 (1x1x1)
b0[:,:,0]
1
Filter W1 (3x3x3)
w1[:,:,0]
0 1 -1
0 -1 0
0 -1 1
w1[:,:,1]
-1 0 0
1 -1 0
1 -1 0
w1[:,:,2]
-1 1 -1
0 -1 -1
1 0 0
Bias b1 (1x1x1)
b1[:,:,0]
0
Output Volume (3x3x2)
o[:,:,0]
2 3 3
3 7 3
8 10 -3
o[:,:,1]
-8 -8 -3
-3 1 0
-3 -8 -5
toggle movement

1.1：卷积层(Convolution Layer)





与全连接网络差别-**Weight Sharing**：

- 每个神经元都查看图像的一个方形块（the receptive field）；
- 神经元从该方形块中的每个像素获取输入并共享权重，而不管它们的方形块在图像中的位置如何，因此是等价的。

特例：D=1

1	1	1	3
4	6	4	8
30	0	1	5
0	2	2	4

图像

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

filters

□ 输出： W2×H2×K;

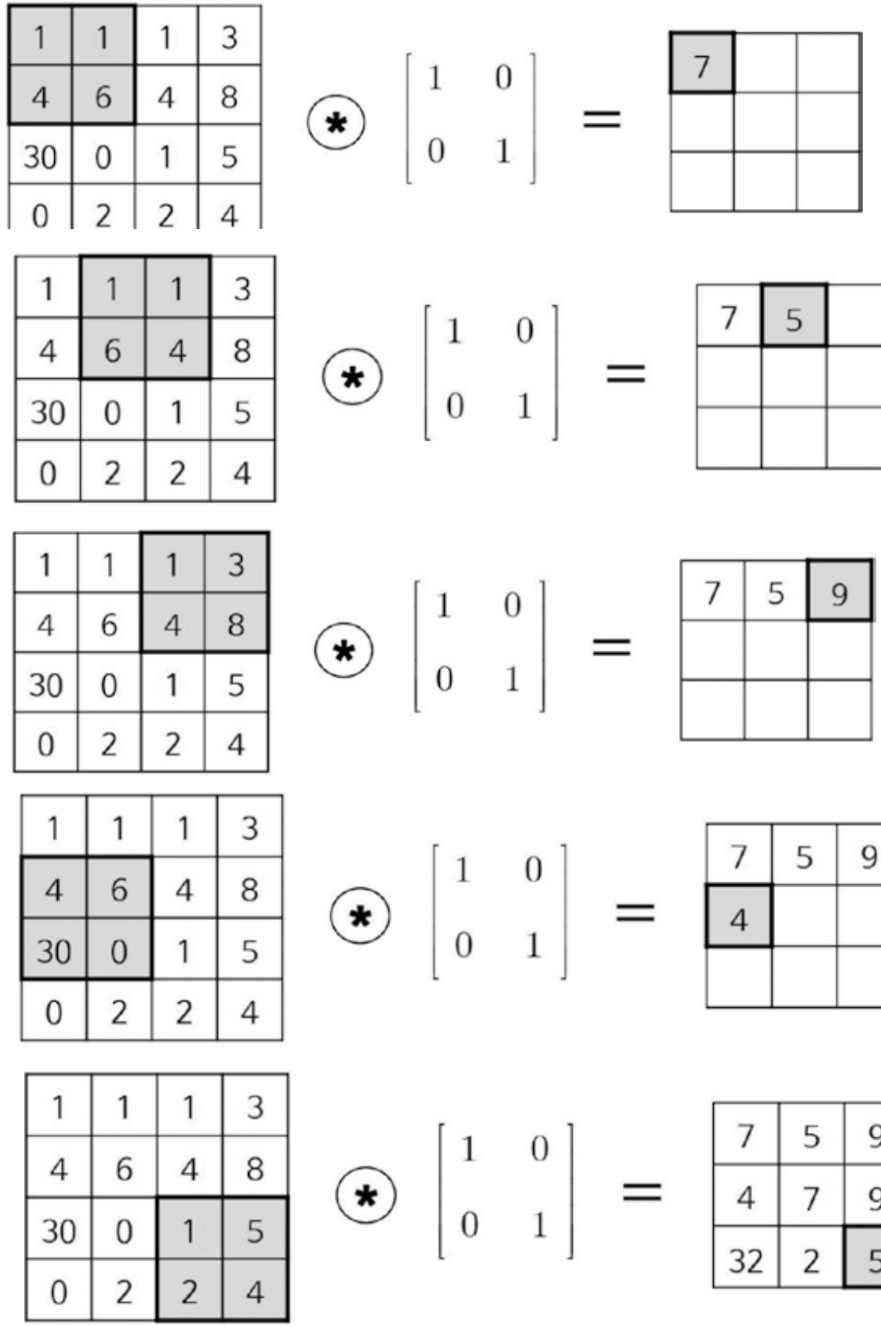
W2=(W1−F+2P)/S+1

=(4-2+2×0)/1+1=3;

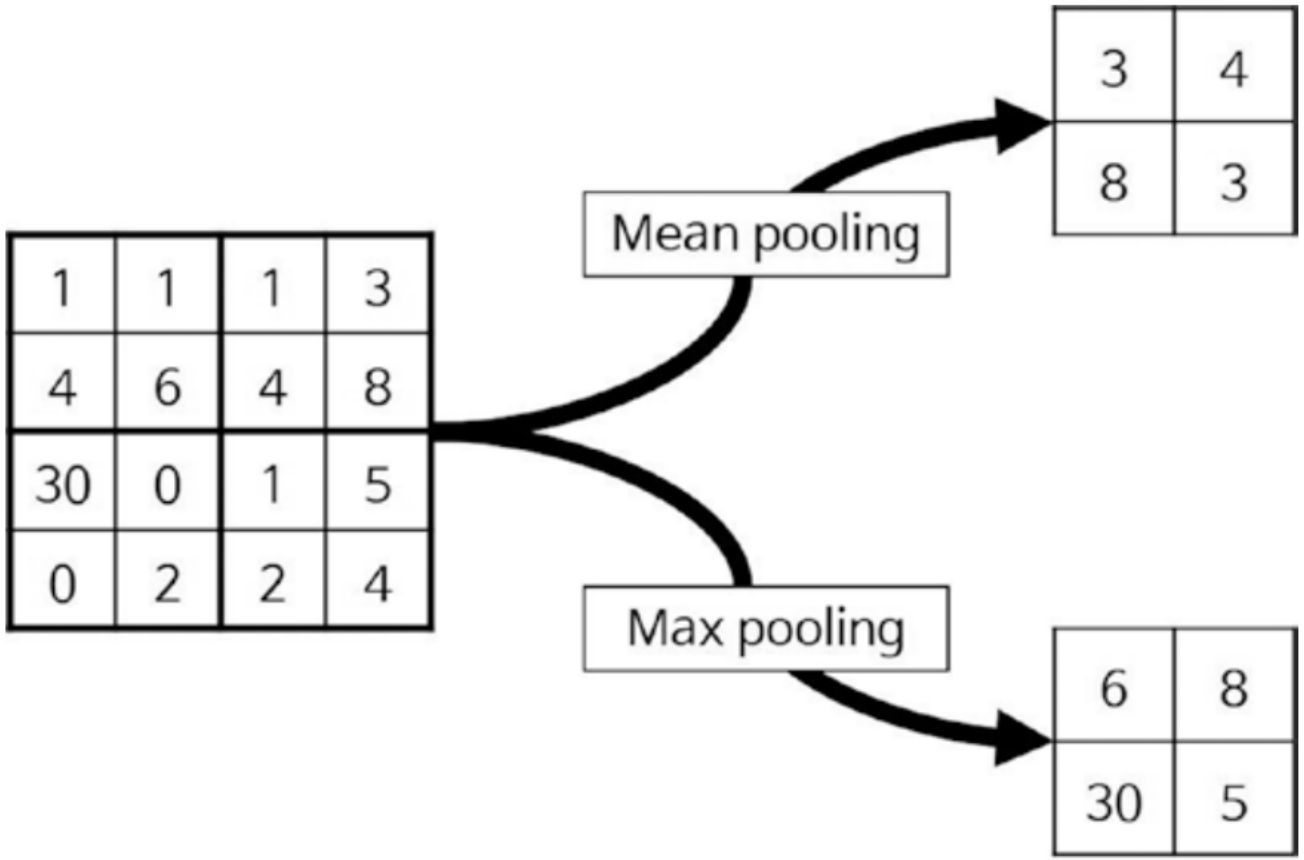
H2=(H1−F+2P)/S+1

=(4-2+2×0)/1+1=3;

K=2;

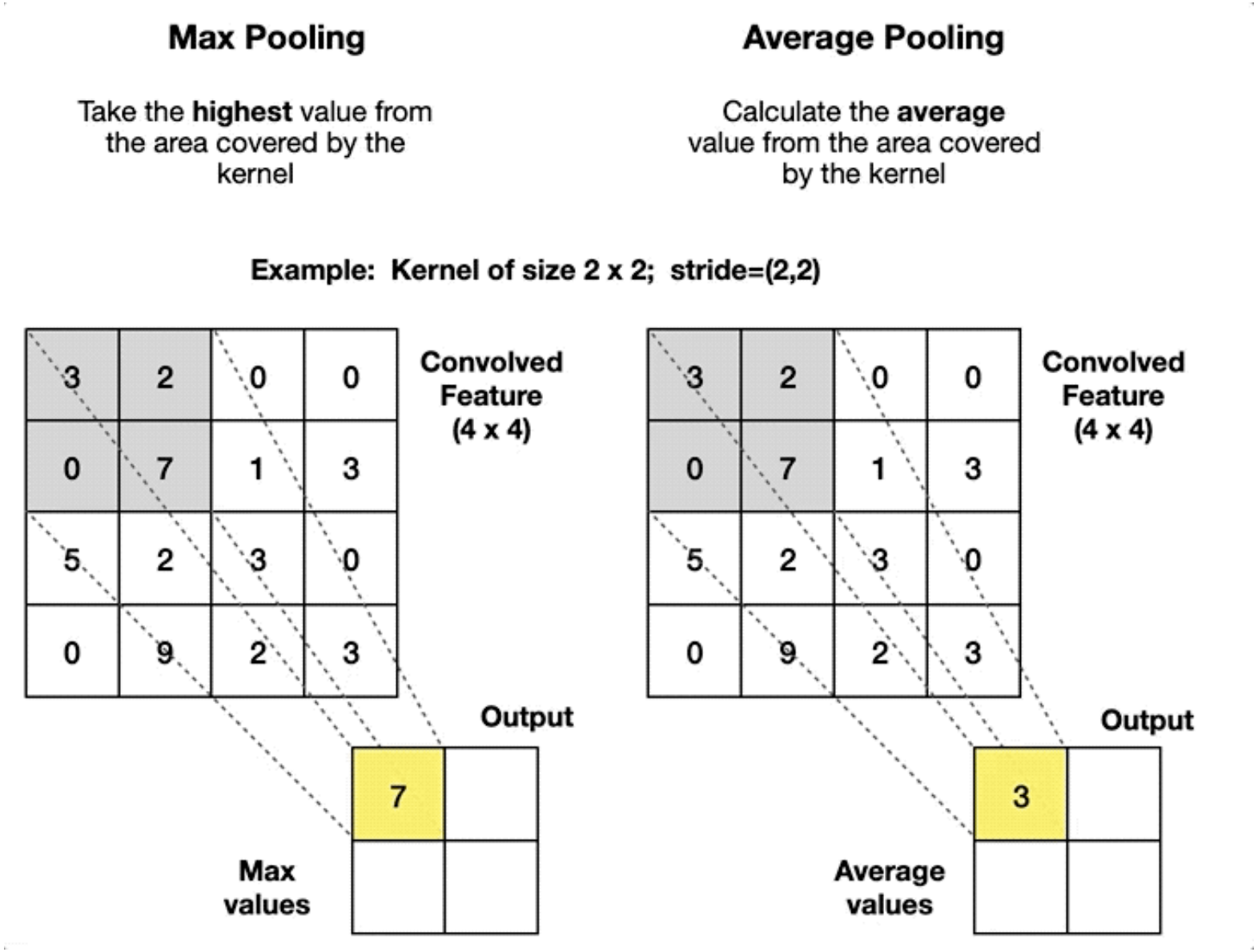


1.2：池化层(Pooling Layer)



平均池化vs最大池化

1.2：池化层(Pooling Layer)



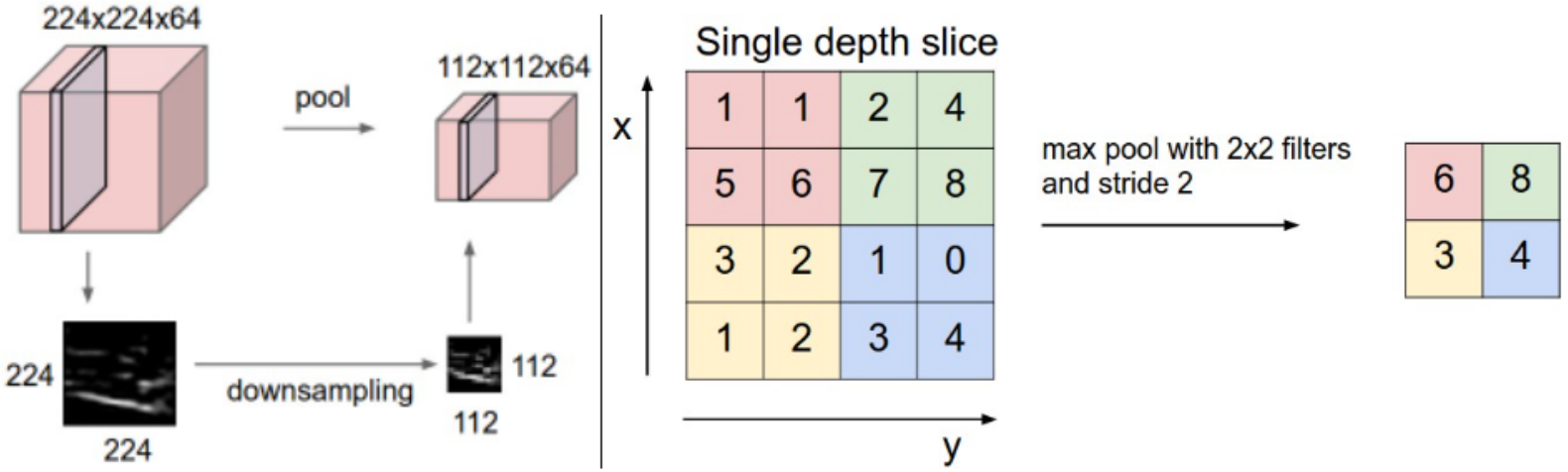
Rafay Qayyum: Introduction To Pooling Layers In CNN

基本特征

- 输入X： *W1×H1×D1*;
filters： *F×F*;
步长Stride: *S*;

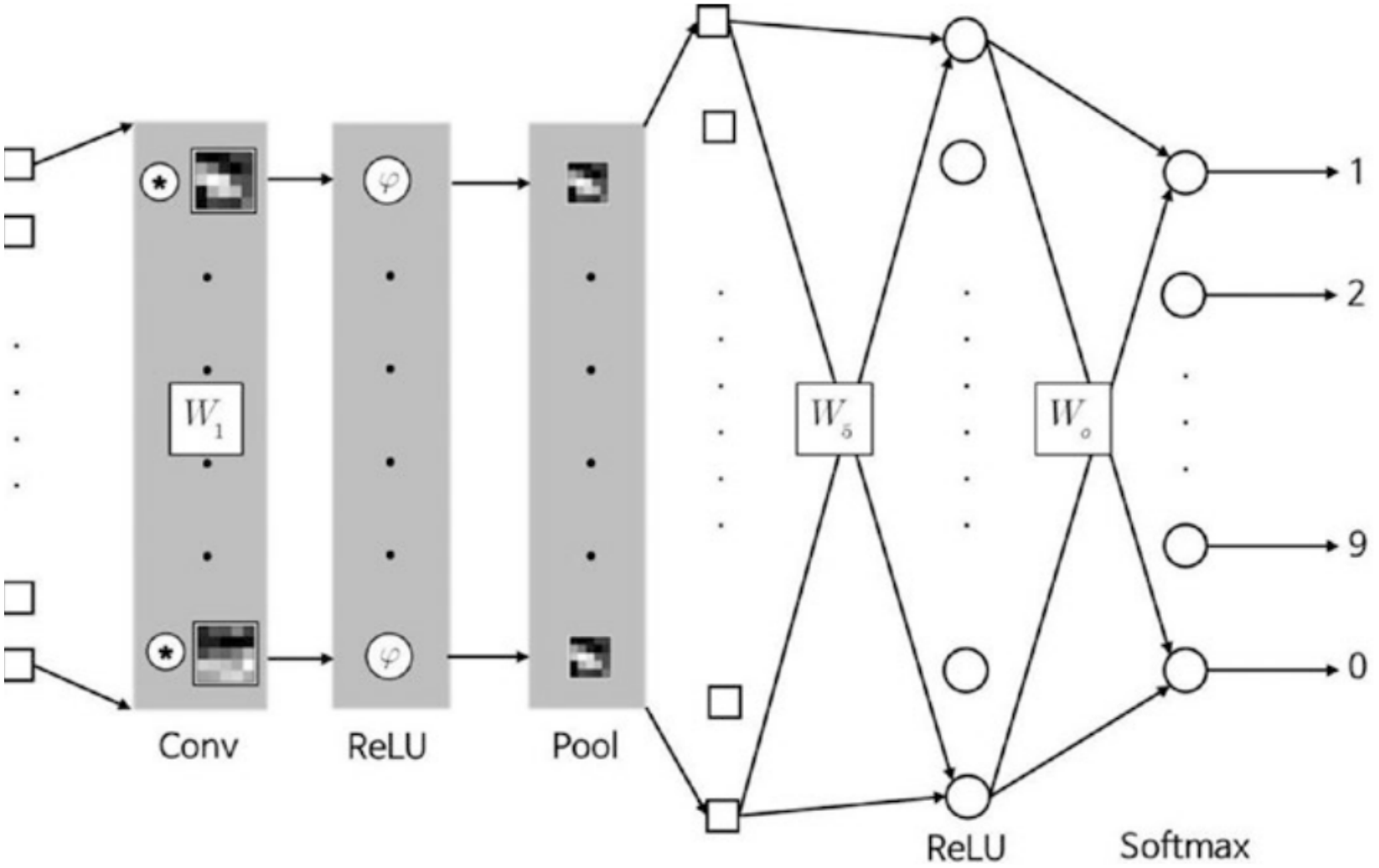
- 输出： W2×H2×*D1*;
W2=(W1−F)/S+1
=(4-2)/2+1=2;

H2=(H1−F)/S+1
=(4-2)/2+1=2;



Source(CS231n: Deep Learning for Computer Vision)

1.3：实例：训练、测试一个卷积神经网络



网络结构（类似LENET）

具体网络结构

P=0,S=1

Layer	Remark	Activation Function
Input	28×28 nodes	-
Convolution	20 convolution filters (9×9)	ReLU
Pooling	1 mean pooling (2×2)	-
Hidden	100 nodes	ReLU
Output	10 nodes	Softmax

训练数据：28*28的MNIST手写数字集（MNIST）



实验7-1：

- 通过给定的Pool、Conv和LoadMnistData函数，通过momentum及 Mini Batch规则，搭建上述卷积神经网络；
- 采用训练数据2000，测试数据300（与训练数据不重合），进行训练和测试，并输出训练后预测结果的准确度（正确的数量 / N=300）。

Step 1: 卷积层定义

```
def Conv(x, W):
    (wrow, wcol, numFilters) = W.shape
    (xrow, xcol)             = x.shape

    yrow = xrow - wrow + 1
    ycol = xcol - wcol + 1

    y = np.zeros((yrow, ycol, numFilters))

    for k in range(numFilters):
        filter = W[:, :, k]
        filter = np.rot90(np.squeeze(filter), 2) # conv2函数需要先做镜像
        y[:, :, k] = signal.convolve2d(x, filter, 'valid')

    return y
```

Step 2: 池化层定义

```python
import numpy as np
from scipy import signal

def Pool(x):
    (xrow, xcol, numFilters) = x.shape
    y = np.zeros((int(xrow/2), int(xcol/2), numFilters))

    for k in range(numFilters):
        filter = np.ones((2,2)) / (2*2)  # 相当于与[1/4 1/4; 1/4 1/4]的filter做卷积;
        image  = signal.convolve2d(x[:, :, k], filter, 'valid')

        y[:, :, k] = image[::2, ::2]

    return y
```

Step 3: MnistConv 函数定义-初始化

```
def MnistConv(W1, W5, Wo, X, D):
    alpha = 0.01
    beta  = 0.95
    momentum1 = np.zeros_like(W1)   # momentum
    momentum5 = np.zeros_like(W5)
    momentumo = np.zeros_like(Wo)
    N = len(D)   # total number of data
    bsize = 10
    blist = np.arange(0, N, bsize) # 0, bsize, bsize*2,....

    for batch in range(len(blist)):   # Loop for all batch
        dW1 = np.zeros_like(W1)
        dW5 = np.zeros_like(W5)
        dWo = np.zeros_like(Wo)

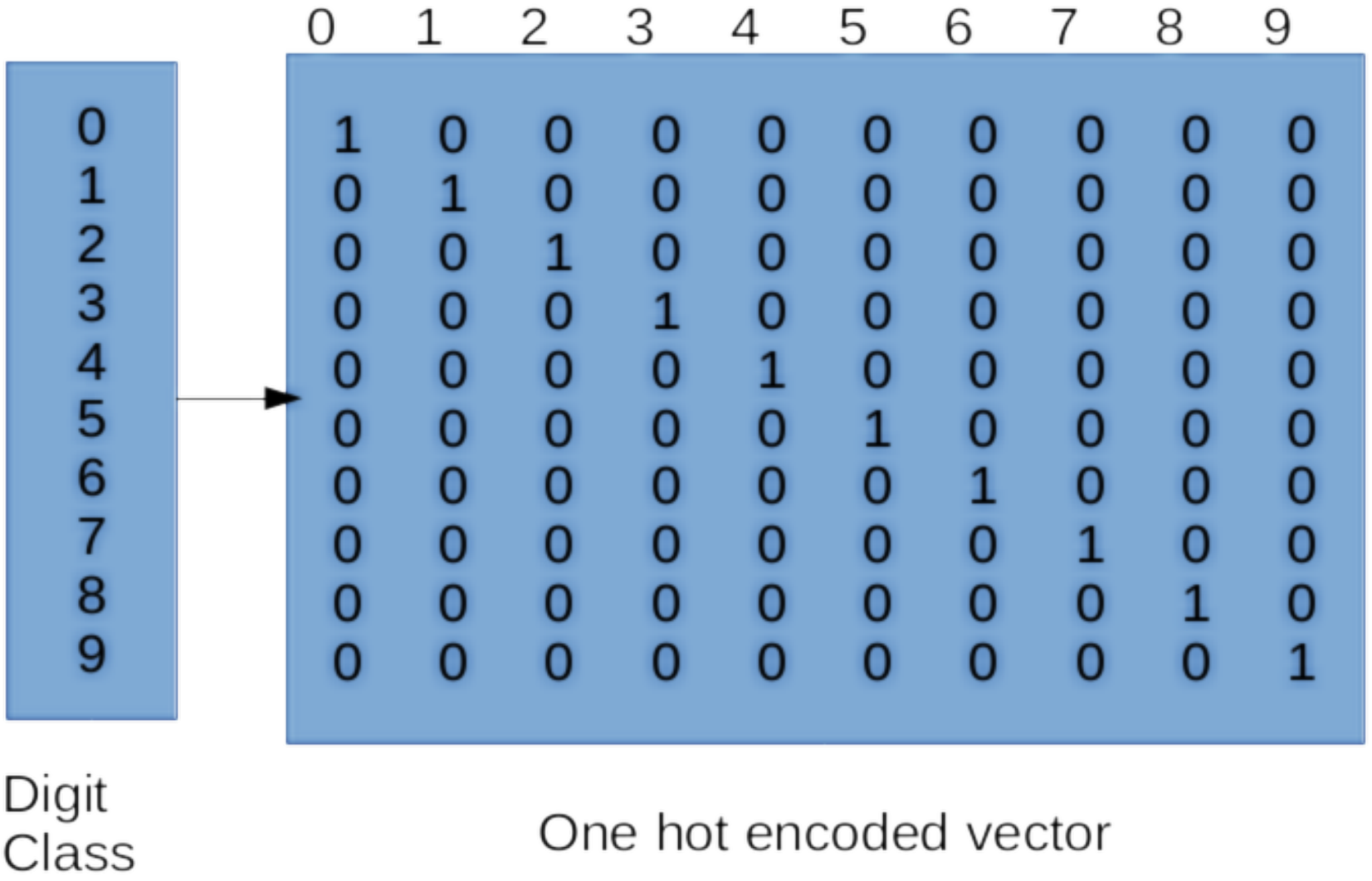
        begin = blist[batch]
```

Step 3: MnistConv 函数定义-正向传输

```
for k in range(begin, begin+bsize):

    x  = XXXXX   #输入
    y1 = XXXXX   #卷积
    y2 = XXXXX   #ReLu激活
    y3 = XXXXX   #池化
    y4 = XXXXX   #卷积全连接转换-Reshape成1维
    v5 = XXXXX   #全连接W5
    y5 = XXXXX   #ReLu激活
    v  = XXXXX   #全连接WO
    y  = XXXXX   #Softmax激活
    # one-hot encoding
    d = np.zeros((10, 1))
    d[D[k][0]][0] = 1
```

解析：Label需要转为one-hot 编码形式：



Step 3: MnistConv 函数定义-反向传输

```
            e     = d - y
            delta = e                          # for softmax
            e5     = np.matmul(Wo.T, delta)     # Output Layer(ReLU)
            delta5 = (y5 > 0) * e5
            e4 = np.matmul(W5.T, delta5)        # HIdden Layer
            e3 = np.reshape(e4, y3.shape)
            e2 = np.zeros_like(y2)              # pooling
            W3 = np.ones_like(y2) / (2*2)
            for c in range(20):
                e2[:, :, c] = np.kron(e3[:, :, c], np.ones((2, 2))) * W3[:, :, c]
            delta2 = (y2 > 0) * e2

            delta1_x = np.zeros_like(W1)
            for c in range(20):
                delta1_x[:, :, c] = signal.convolve2d(x[:, :], np.rot90(delta2[:, :, c], 2),
'valid')
```

Step 3: MnistConv 函数定义-更新momentum

```
            dW1 = dW1 + delta1_x
            dW5 = dW5 + np.matmul(delta5, y4.T)
            dWo = dWo + np.matmul(delta, y5.T)

        dW1 = dW1 / bsize
        dW5 = dW5 / bsize
        dWo = dWo / bsize

        momentum1 = XXXXXX
        W1        = W1 + momentum1

        momentum5 = XXXXXX
        W5        = W5 + momentum5

        momentumo = XXXXXX
        Wo        = Wo + momentumo
```

Step 4: LoadMnistData函数使用格式：

```
from LoadMnistData import *

Images, Labels = LoadMnistData('MNIST\\t10k-images-idx3-ubyte.gz',
'MNIST\\t10k-labels-idx1-ubyte.gz')
Images = np.divide(Images, 255)  #矩阵的对应元素相除,归一化
X = Images[0:2000, :, :]
D = Labels[0:2000]
```

Step 5：初始化权重W，调用MnistConv训练，取出2000:2300的数据测试准确率